

WEEK-1 SKILL

1. To Install Linux VM, Configure GCC, Setup Git Repository, Create Project Structure, Understand Shell Architecture, Build Initial Makefile.

```
Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading:
the_boss@DESKTOP-I8C8AJG:~/oss$ tree
.
├── LICENSE
├── Makefile
├── README.md
├── assets
│   └── architecture.png
├── bin
├── docs
│   ├── design.md
│   └── report.pdf
├── include
│   ├── builtin.h
│   ├── executor.h
│   ├── parser.h
│   └── shell.h
├── obj
├── scripts
│   └── run.sh
├── src
│   ├── builtin.c
│   ├── executor.c
│   ├── main.c
│   ├── parser.c
│   └── shell.c
└── tests
    ├── test_executor.c
    └── test_parser.c

9 directories, 18 files
```

Code:

```
the_boss@DESKTOP-I8C8AJG: x + v
GNU nano 8.7.1 fork_example.c
#include <stdio.h>
#include <unistd.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("This is the Child Process.\n");
        printf("Child PID = %d\n", getpid());
    }
    else {
        printf("This is the Parent Process.\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }

    return 0;
}
```

OUTPUT:

```
the_boss@DESKTOP-I8C8AJG:~/oosp$ nano fork_example.c
the_boss@DESKTOP-I8C8AJG:~/oosp$ gcc fork_example.c
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./fork_example
-bash: ./fork_example: No such file or directory
the_boss@DESKTOP-I8C8AJG:~/oosp$ ./a.out
This is the Parent Process.
Parent PID = 1556
Child PID = 1557
This is the Child Process.
Child PID = 1557
```

REPORT: I successfully installed the Linux environment and configured the GCC compiler. I created a Git repository and organized the required project folders and files. I also understood

2.To Analyze process abstraction, execute fork(), understand exec() family, analyze parent-child relationships, inspect process tree, practice system call tracing.

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed!\n");
    }
    else if (pid == 0) {
        printf("Child Process\n");
        execlp("ls", "ls", "-l", (char *)NULL);
    }
    else {
        printf("Parent Process\n");
        wait(NULL);
    }

    return 0;
}
```

OUTPUT:

```

the_boss@DESKTOP-I8C8AJG:~/oosp$ nano fork_exec.c
the_boss@DESKTOP-I8C8AJG:~/oosp$ gcc fork_exec.c -o fork_exec
./fork_exec
Parent Process
Child Process
total 72
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 LICENSE
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 Makefile
-rw-r--r-- 1 the_boss the_boss 0 Aug 11 15:05 README.md
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:17 a.out
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 assets
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 bin
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 docs
-rw-r--r-- 1 the_boss the_boss 443 Aug 11 15:14 fork_example.c
-rwxr-xr-x 1 the_boss the_boss 16080 Aug 11 15:22 fork_exec
-rw-r--r-- 1 the_boss the_boss 373 Aug 11 15:22 fork_exec.c
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 include
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 obj
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 scripts
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 src
drwxr-xr-x 2 the_boss the_boss 4096 Aug 11 15:05 tests

```

REPORT: I executed the fork() program and observed how a parent process creates a child process. I understood the working of the exec() family and learned how processes are replaced during execution. I also examined the process tree and practiced tracing system calls to understand how a program interacts with the operating system.

Session 2

Task-1 To Create Main Loop, Display Prompt, Read User Input, Handle Exit Conditions, Design Control Flow Diagram, Test Interactive Loop

```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 shell.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    char input[100];

    while (1)
    {
        printf("mysh> ");
        fgets(input, sizeof(input), stdin);

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
        {
            printf("Exiting shell...\n");
            break;
        }

        if (strlen(input) == 0)
            continue;

        printf("You entered: %s\n", input);
    }

    return 0;
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG:~/oss$ gcc shell.c -o bin/shell
the_boss@DESKTOP-I8C8AJG:~/oss$ ./bin/shell
mysh> hello
You entered: hello
mysh> exit
Exiting shell...
the_boss@DESKTOP-I8C8AJG:~/oss$ |
```

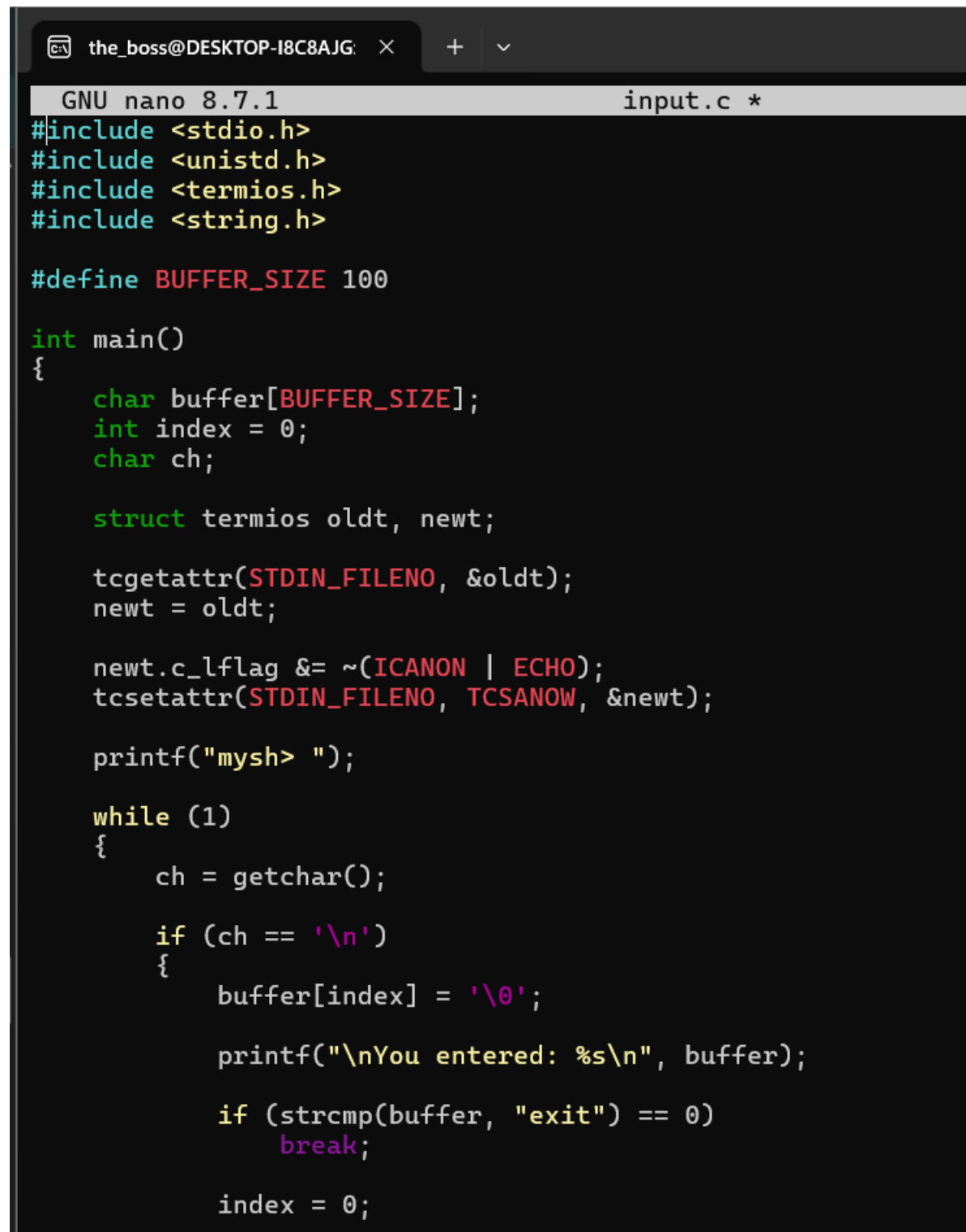
Report:

I created a simple interactive shell using a main loop. The program displays the `mysh>` prompt and takes input from the user. It checks the entered input and displays the command entered by the user. I also added an exit condition using the `exit` command. I

tested the program with different inputs and verified that the interactive loop works correctly.

Task-2:

To Capture Keyboard Input, Handle Backspace, Process Enter Key, Manage Input Buffer, Support Multi-Character Commands, Test User Interaction



```
the_boss@DESKTOP-I8C8AJG: × + v
GNU nano 8.7.1 input.c *
#include <stdio.h>
#include <unistd.h>
#include <termios.h>
#include <string.h>

#define BUFFER_SIZE 100

int main()
{
    char buffer[BUFFER_SIZE];
    int index = 0;
    char ch;

    struct termios oldt, newt;

    tcgetattr(STDIN_FILENO, &oldt);
    newt = oldt;

    newt.c_lflag &= ~(ICANON | ECHO);
    tcsetattr(STDIN_FILENO, TCSANOW, &newt);

    printf("mysh> ");

    while (1)
    {
        ch = getchar();

        if (ch == '\n')
        {
            buffer[index] = '\0';

            printf("\nYou entered: %s\n", buffer);

            if (strcmp(buffer, "exit") == 0)
                break;

            index = 0;
        }
    }
}
```

Output:

```
the_boss@DESKTOP-I8C8AJG: ~/oss$ gcc input.c -o bin/mysh
the_boss@DESKTOP-I8C8AJG: ~/oss$ ./bin/input
mysh> hello world
You entered: hello world
mysh> exit
You entered: exit

Exiting shell...
the_boss@DESKTOP-I8C8AJG: ~/oss$ |
```

Report:

I created a program to capture keyboard input character by character. I used an input buffer to store the characters entered by the user. I implemented Backspace to delete the last entered character and Enter to process the complete input. I also tested multi-character commands and the `exit` command. The program worked successfully during testing.